

PRACTICE EXERCISES OF THE MICROPROCESSORS &
MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Văn Nhật Tân

Student code: 24521587

PRACTICE EXERCISE #5:

ADDITION OF TWO 32-BIT NUMBERS ON THE 8086 PROCESSOR

1. Thực thi example ADD/SUB.

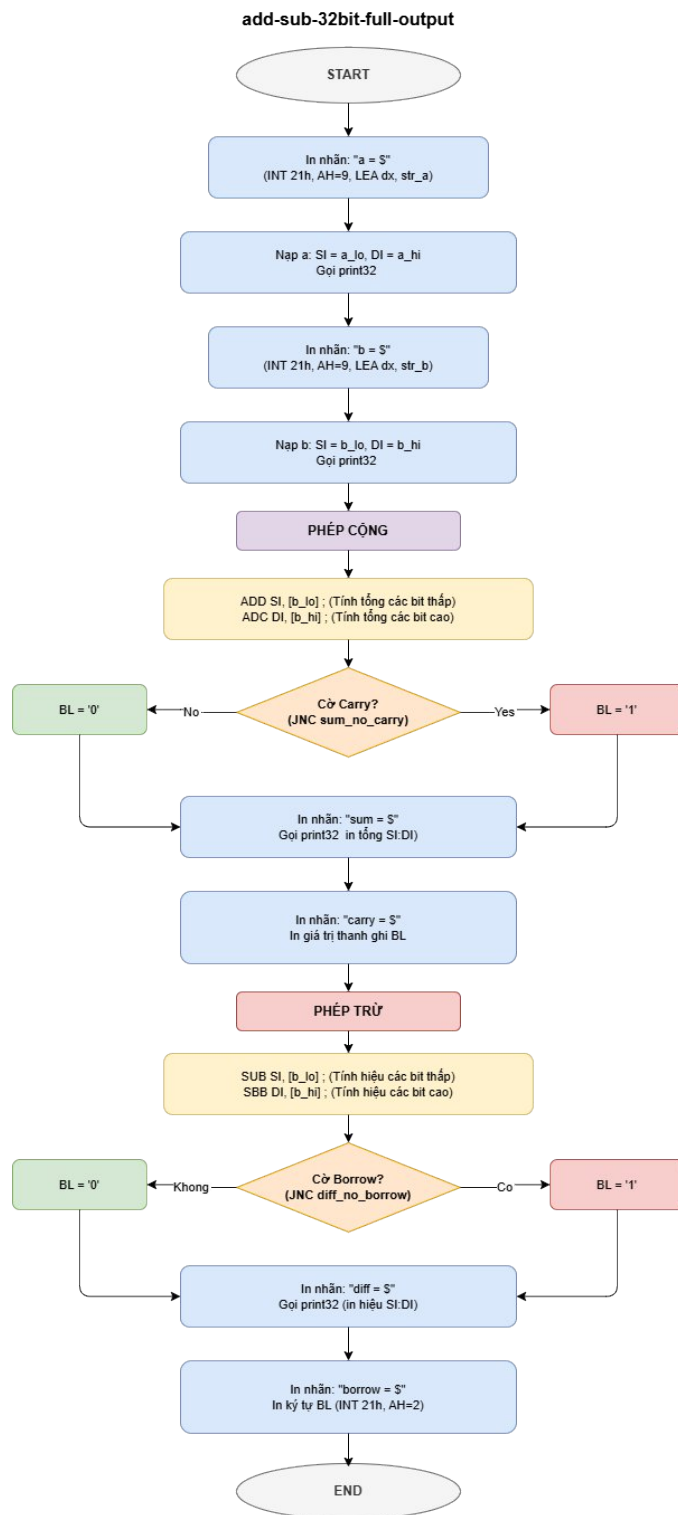
a. Source Code

ASSEMBLY	Giải Thích
<pre>name "add-sub" org 100h mov al, 5 ; bin=00000101b mov bl, 10 ; hex=0ah or bin=00001010b ; 5 + 10 = 15 (decimal) or hex=0fh or bin=00001111b add bl, al ; 15 - 1 = 14 (decimal) or hex=0eh or bin=00001110b sub bl, 1 ; print result in binary: mov cx, 8 print: mov ah, 2 ; print function. mov dl, '0' test bl, 10000000b ; test first bit. jz zero mov dl, '1'</pre>	Khai báo và gán giá trị cho thanh ghi al (5) và bl (10). Sử dụng add và sub để thực hiện tính toán, kết quả của các phép toán lần lượt được ghi vào thanh ghi bl. mov cx, 8: Thiết lập biến đếm cho vòng lặp là 8 (vì thanh ghi BL có kích thước 1 byte = 8 bit). print: mov ah, 2: Nhãn print đánh dấu điểm bắt đầu của vòng lặp. AH = 2 là lệnh chuẩn bị gọi ngắt DOS để in một ký tự (ký tự này sẽ được lấy từ thanh ghi DL). mov dl, '0': Đặt mặc định ký tự chuẩn bị in ra là chữ số '0'. test bl, 10000000b: Đây là bước quan trọng nhất. Phép TEST thực hiện phép toán AND logic giữa BL và 10000000b (chỉ có bit ngoài cùng bên trái là 1). jz zero: Nếu bit ngoài cùng bên trái của BL là 0, chương trình nhảy

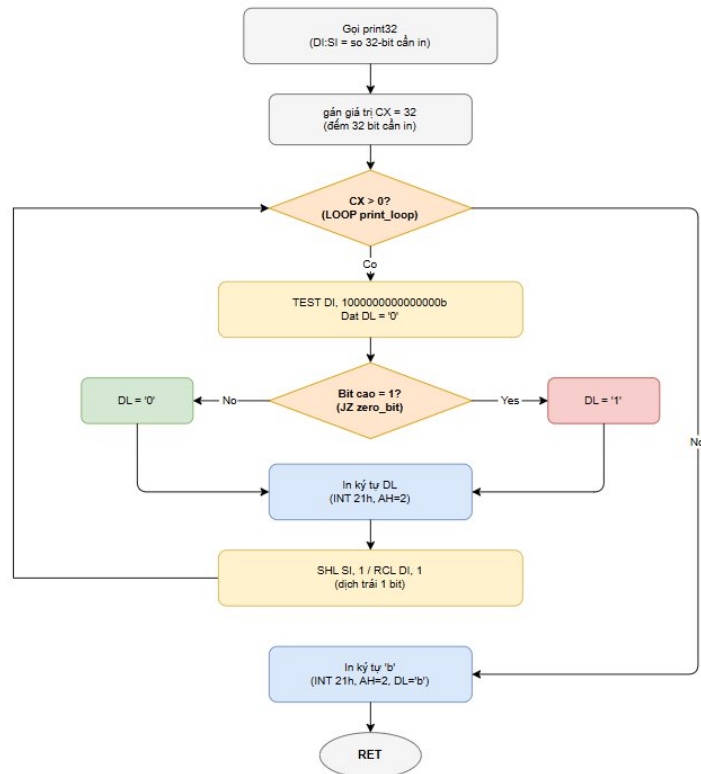
<pre> zero: int 21h shl bl, 1 loop print ; print binary suffix: mov dl, 'b' int 21h ; wait for any key press: mov ah, 0 int 16h ret </pre>	thẳng đến nhãn zero để in ra ký tự '0'. mov dl, '1': Nếu lệnh jz không xảy ra chương trình chạy tiếp dòng này, thay đổi ký tự chuẩn bị in thành chữ số '1'. zero: int 21h: Gọi ngắt hệ thống DOS (Interrupt 21h). Lúc này nó sẽ in ký tự đang chứa trong DL ('0' hoặc '1') ra màn hình. shl bl, 1: Dịch toàn bộ các bit trong BL sang trái 1 vị trí. Việc này giúp đẩy bit tiếp theo lên vị trí ngoài cùng bên trái để vòng lặp sau có thể tiếp tục kiểm tra. loop print: Tự động giảm CX đi 1. Nếu CX vẫn lớn hơn 0, nó sẽ quay lại nhãn print để kiểm tra và in bit tiếp theo. Quá trình này lặp lại 8 lần.
---	--

2. Viết chương trình thực hiện cộng trừ 2 số 32bit.

a. Flowchart.



Flowchart chương trình Cộng/Trừ 2 số 32 bit.



Flowchart hàm print32

b. Nguyên lý hoạt động

Chương trình thực hiện phép cộng và phép trừ hai số 32-bit bằng Assembly 8086, sau đó in kết quả dưới dạng nhị phân 32-bit.

Do vi xử lý 8086 chỉ xử lý dữ liệu 16-bit nên mỗi số 32-bit được chia thành hai phần:

a_hi : 16 bit cao

a_lo : 16 bit thấp

Tương tự cho số b.

Giá trị của 2 số a, b được cài đặt ngay trong code, a = 10; b = 15. Hai thanh ghi được dùng để xử lý dữ liệu: SI chứa 16 bit thấp, DI chứa 16 bit cao.

- In giá trị của a và b

Chương trình dùng ngắt DOS int 21h để in chuỗi ra màn hình:

```
mov ah, 9
```

```
lea dx, str_a
```

```
int 21h
```

Sau đó đưa giá trị vào thanh ghi:

```
mov si, [a_lo]
```

```
mov di, [a_hi]
```

và gọi thủ tục print32 để in số ở dạng nhị phân 32-bit.

- **Phép cộng 32-bit**

16 bit thấp được cộng trước:

add si, [b_lo]

Sau đó cộng 16 bit cao kèm cờ nhớ bằng lệnh:

adc di, [b_hi]

ADC giúp cộng thêm Carry Flag nếu phần thấp bị tràn.

Sau phép cộng, chương trình kiểm tra carry:

jnc sum_no_carry

Nếu có carry thì in 1, ngược lại in 0.

Kết quả:

$$10 + 15 = 25$$

- **Phép trừ 32-bit**

16 bit thấp được trừ trước:

sub si, [b_lo]

Sau đó trừ 16 bit cao kèm cờ mượn:

sbb di, [b_hi]

SBB sẽ trừ thêm Borrow Flag nếu phần thấp phải mượn.

Chương trình kiểm tra borrow bằng:

jnc diff_no_borrow

Kết quả:

$$10 - 15 = -5$$

Số âm được biểu diễn theo dạng bù 2:

11111111111111111111111111111111011b

- **Xuất ra màn hình bằng thủ tục print32**

Thủ tục print32 dùng để in số 32-bit dưới dạng nhị phân.

Chương trình lặp 32 lần:

mov cx, 32

Mỗi lần lặp sẽ kiểm tra bit cao nhất của DI:

test di, 1000000000000000b

Nếu bit bằng 1 thì in ký tự '1', ngược lại in '0'.

Sau đó dịch trái cặp thanh ghi:

shl si, 1

rcl di, 1

để tiếp tục kiểm tra bit kế tiếp. Tương tự với hàm print trong example.

Kết quả hiển thị:

a = 000000000000000000000000000000001010b

[illegible]

sum = 0000000000000000000000000000000011001b (carry = 0)

`diff = 11111111111111111111111111111011b (borrow = 1)`

c. Source code

ASSEMBLY

```
name "add-sub-32bit-full-output"
```

```
org 100h
```

```
jmp start
```

```
a_hi    dw 0000h
```

a lo dw 10

b hi dw 0000h

b lo dw 15

```
str_a db "a" = $"
```

```
str_b db 13, 10, "b" = "$"
```

```
str_sum db 13, 10, "sum  = $"
```

```
str diff db 13, 10, "diff = $"
```

```
str_carry db " (carry = $"
```

```
str bor db " (borrow = $"
```

```
str_end db ")$"
```

start:

```
mov ah, 9
```

lea dx, str a

int 21h

```
mov si, [a_lo]
```

```
mov di, [a_hi]
```

```
call print32
```

```
mov ah, 9
```

```
lea dx, str_b
int 21h
mov si, [b_lo]
mov di, [b_hi]
call print32
```

```
mov si, [a_lo]
mov di, [a_hi]
add si, [b_lo]
adc di, [b_hi]
```

```
mov bl, '0'
jnc sum_no_carry
mov bl, '1'
sum_no_carry:
```

```
mov ah, 9
lea dx, str_sum
int 21h
```

```
call print32
```

```
mov ah, 9
lea dx, str_carry
int 21h
mov ah, 2
mov dl, bl
int 21h
mov ah, 9
lea dx, str_end
int 21h
```

```
mov si, [a_lo]
mov di, [a_hi]
sub si, [b_lo]
```

```

    sbb di, [b_hi]

    mov bl, '0'
    jnc diff_no_borrow
    mov bl, '1'
diff_no_borrow:

    mov ah, 9
    lea dx, str_diff
    int 21h

    call print32

    mov ah, 9
    lea dx, str_bor
    int 21h
    mov ah, 2
    mov dl, bl
    int 21h
    mov ah, 9
    lea dx, str_end
    int 21h

    mov ah, 0
    int 16h
    ret

print32 proc
    mov cx, 32
print_loop:
    mov ah, 2
    mov dl, '0'

    test di, 1000000000000000b
    jz zero_bit

```



```
        mov dl, '1'
zero_bit:
        int 21h

        shl si, 1
        rcl di, 1

        loop print_loop

        mov ah, 2
        mov dl, 'b'
        int 21h
        ret
print32 endp
```

3. Video mô phỏng chương trình và source code.

https://drive.google.com/drive/folders/1cA_PDEoY9cUkAPM6yN70NNV4XCG28DW9?usp=sharing